

Backend Odaklı Küçük Programlama Dili

AI modelleri için güvenli, insan için doğal, native performansa yakın küçük bir dil tasarımı
ve geliştirme rehberi

Tasarım hedefi

Bu doküman bir akademik dil teorisi metni değil; backend üretimi, AI destekli kod üretimi, güvenli low-level performans ve küçük compiler inşası için uygulanabilir bir teknik yol haritasıdır.

Alan	Karar
Çalışma adı	Arelang / küçük backend dili
Yakın akrabalar	Go ergonomisi, Zig açıklığı, Rust güvenlik sınırı, pslang/C ABI sadeliği
Ana hedef	AI tarafından az tokenla, az hata ile üretilebilen yüksek performanslı backend kodu
İlk wedge	HTTP servisleri, workerlar, CLI, sandboxed plugin, deployment agent
Bilinçli kapsam dışı	Genel amaçlı C++ replacement, oyun motoru dili, ML training dili, browser frontend dili

Sürüm: 0.1 teknik tasarım taslağı - Hazırlanma tarihi: 26 Mayıs 2026

İçindekiler

1. Yönetici özeti
2. Problemin tanımı
3. Kaynak bulgular ve tasarım ilkeleri
4. Dilin ürün tanımı
5. Syntax ve yüzey tasarımı
6. Type system
7. Memory modeli
8. Error handling
9. Backend standard library
10. Concurrency ve worker modeli
11. Modül ve paket sistemi
12. Compiler mimarisi
13. IR ve optimizasyonlar
14. AI-agent uyumluluğu
15. Güvenlik ve sandbox
16. Örnek backend uygulaması
17. MVP geliştirme yol haritası
18. Test stratejisi
19. Kaçınılacak özellikler
20. Açık kararlar ve gelecek işler
21. Kaynak notları

1. Yönetici özeti

Bu dokümanın önerdiği dil, klasik anlamda yeni bir genel amaçlı dil yaratma girişimi değildir. Daha dar ve daha gerçekçi bir hedefe sahiptir: backend odaklı, küçük, hızlı derlenen, native performansla yakın çalışan, AI modelleri tarafından tutarlı şekilde üretilebilen ve insanlar tarafından doğal öğrenilebilen bir sistem dili. Dilin ana değeri, C++ kadar geniş ve tehlikeli bir yüzey alanı sunmadan; Go kadar sade, Zig kadar açık, Rust kadar güvenlik bilinci olan bir üretim deneyimi sağlamaktır.

Bir dilin AI için iyi olması yalnızca kısa syntax anlamına gelmez. Asıl mesele, modelin tahmin etmek zorunda kaldığı gizli davranışları azaltmaktır. Implicit cast, hidden allocation, exception flow, macro expansion, overload resolution, undefined behavior, header/build karmaşası ve karmaşık lifetime kuralları AI destekli geliştirmede hata oranını artırır. Bu nedenle ideal dilin küçük, deterministik, açık ve compiler tarafından iyi denetlenebilir olması gerekir.

Bu tasarımın backend odağı bilinçlidir. HTTP server, routing, JSON, validation, config, logging, metrics, tracing, health check, worker pool, queue adapter, database interface, auth primitives ve test araçları dilin standard library tarafında birinci sınıf olmalıdır. Token ekonomisi sadece syntax kısaltarak değil, AI modeline sürekli framework glue code yazdırmayarak kazanılır.

Belge boyunca önerilen dil için çalışma adı olarak Arelang kullanılacaktır. İsim nihai değildir. Önemli olan isim değil, tasarım çizgisidir: küçük çekirdek, güçlü backend prelude, nullsız tip sistemi, explicit error propagation, scoped/arena-first bellek modeli, sınırlı generics, structured compiler diagnostics ve güvenli sandbox yetenekleri.

Temel hüküm

Bu dil C++ yerine her şeyi yazmak için değil; AI tarafından üretilecek güvenli, hızlı, küçük backend servisleri ve sistem bileşenleri için tasarlanmalıdır.

1.1 Başarı kriterleri

Kriter	Anlamı
Kod üretim doğruluğu	Modelin ürettiği kodun ilk derleme denemesinde veya tek düzeltme döngüsünde çalışması.
Token verimliliği	Aynı backend endpointi için Go/TypeScript eşdeğerinden daha az ceremony ve daha az framework konfigürasyonu.
Performans	C/Zig seviyesine yakın native binary; düşük latency; düşük bellek overheadi; opsiyonel runtime.
Güvenlik	Null pointer yok, implicit cast yok, data race ve arena escape hataları compiler tarafından yakalanır.
Öğrenilebilirlik	Yeni başlayan biri 1 günde syntax, 1 haftada backend servis, 1 ayda compiler katkısı anlayabilmeli.

2. Problemin tanımı

AI modelleri Python, TypeScript ve Go gibi dilleri genellikle daha az hatayla yazar; fakat bu dillerin her biri low-level performans, bellek kontrolü veya native deployment açısından sınırlara sahiptir. C++ ve Rust ise performanslıdır, fakat AI üretimi açısından daha fazla semantik risk taşır. C++ çok fazla implicit davranışa, tarihsel karmaşaya ve undefined behavior alanına sahiptir. Rust daha güvenlidir fakat borrow checker, lifetimes, async ve trait sistemi LLM kod üretimini zorlaştırabilir.

Backend geliştirmede asıl sorun yalnızca runtime performansı değildir. Tipik bir servis endpointi auth, validation, JSON, DB, logging, metrics, tracing, error mapping, timeout, retry, queue ve test katmanlarıyla birlikte gelir. AI bu katmanları farklı frameworklere göre tekrar tekrar üretmek zorunda kaldığında hem token tüketimi artar hem de integration hataları çoğalır.

Bu nedenle yeni dilin görevi şudur: low-level performans yeteneği verirken, backend için gerekli yüksek seviyeli rutinleri standart ve güvenli hale getirmek. Dil C gibi hafif olmalı ama C gibi tehlikeli olmamalı; Go gibi sade olmalı ama Go kadar GC/runtime bağımlı olmamalı; Rust gibi güvenli olmalı ama Rust kadar öğrenme ve üretim yükü bindirmemelidir.

2.1 Tasarım gerilimi

Gerilim	Bir taraf	Diğer taraf	Önerilen çözüm
Performans	Manual control ister	Manual control AI için hata üretir	Safe defaults + explicit unsafe
Sadelik	Az özellik ister	Backend pratikliği stdlib ister	Küçük çekirdek + opinionated backend lib
Güvenlik	Katı kurallar ister	Fazla katılık üretim hızını düşürür	Scoped ownership + arena
Token ekonomisi	Kısa syntax ister	Aşırı sembolik syntax öğrenmeyi zorlaştırır	Kısa ama açık keywordler
Interop	C ABI ister	C semantiği tehlikeli olabilir	FFI boundary policy ve

		capability
--	--	------------

3. Kaynak bulgular ve tasarım ilkeleri

Lisyrus tarafından yazılan kendi programlama dilini yapma yazısı, bu dokümanın tasarım düşüncesiyle önemli kesişimler içerir. Yazıdaki dil; imperative, eager, call-by-value, low-level, static, strict ve nominal type system kullanan küçük bir dil olarak kurgulanmıştır. Hedef C++ veya Rust öldürmek değil, modlama ve low-level kullanım için küçük ve pratik bir dil geliştirmektir.

Yazıda özellikle dört karar bizim tasarımıımız açısından değerlidir: C interop önemli bir hedef olarak görülür; compiler küçük parçalara ayrılır; implicit type cast reddedilir; küçük type system ile gerçek program yazılabileceği gösterilir. Ayrıca parser, AST, IR, interpreter ve JIT ayrımı yeni dil geliştirmek için pratik bir mimari önerir.

Buna karşılık backend ve AI odaklı bir dil için bazı seçimleri değiştirmek gerekir. Indentation-based scope insanlar için temiz görünse de patch üretimi, formatter, diff ve LLM düzeltme döngüsü açısından brace tabanlı ve zorunlu formatterlı syntax daha güvenlidir. Null literal ise pointer dünyasında kullanışlı görünse de AI üretimi için risklidir; onun yerine Option tipi veya T? modeli tercih edilmelidir. C++ tarzı unrestricted templates ve overload yaklaşımı da hata mesajlarını karmaşıklştıracağı için sınırlı protocol/trait sistemine yönelmek daha uygundur.

Bulgu	Alınacak mı?	Gerekçe
Küçük type system	Evet	Aynı şekilde korunmalı.
C ABI ve foreign function	Evet	Backend ve sistem entegrasyonu için gerekli.
No implicit casts	Evet	AI için kritik güvenlik kuralı.
Indentation scope	Kısmen hayır	Brace + formatter daha güvenli.
Null literal	Hayır	Option/T? ile değiştirilmeli.
Unrestricted templates	Hayır	Sınırlı generics/protocol daha iyi.
Manual alloc/free	Kısmen hayır	Arena-first memory modeli daha güvenli.

3.1 On tasarım emri

İlke	Açıklama
Gizli davranış yok	Allocation, cast, IO, network, async switch, blocking call ve unsafe boundary görünür olmalıdır.
Null yok	Yokluk explicit Option/T? ile temsil edilir; pointer ile absence aynı kavram değildir.
Exception yok	Hata dönüşleri tip imzasında görünür ve ? operatörüyle propagasyon yapılır.
Varsayılan immutable	Mutable state açıkça işaretlenir; paylaşılan mutable state daha da açık işaretlenir.
Arena-first allocation	Kısa ömürlü backend nesneleri request/job/task scope içinde toplanır.
Sınırlı generics	Derlenebilirlik ve hata mesajları generic gücünden daha önceliklidir.
Tek formatter	Stil tartışması ve LLM'in farklı format üretmesi engellenir.
Structured diagnostics	Compiler hataları hem insan hem AI tarafından parse edilebilir olmalıdır.
Stdlib güçlü	Backend için temel ihtiyaçlar üçüncü parti framework karmaşasına bırakılmaz.
Unsafe izole	Performans kapısı var ama policy, raporlama ve dosya/fonksiyon seviyesinde izolasyonla sınırlanır.

4. Dilin ürün tanımı

Arelang bir backend/system utility dilidir. İlk hedefi büyük GUI uygulamaları, mobil uygulamalar, ML training pipeline'ları veya browser frontend değildir. İlk hedef, AI destekli olarak üretilebilecek küçük ama ciddi servislerdir: HTTP API, internal tool, worker, deployment agent, CLI, sandboxed workflow runner, parser, data transformer ve gateway.

Dil iki kullanıcı grubuna aynı anda hitap eder. İnsan kullanıcı için doğal öğrenilirlik önemlidir: az keyword, tutarlı syntax, net hata mesajları ve backend primitive'leri. AI modeli için deterministik üretilebilirlik önemlidir: az grammar alternatifi, düşük implicit davranış, kısa ama açık kod, structured compiler feedback ve standart proje yapısı.

Dilin ürün stratejisi "önce küçük ve doğru" olmalıdır. İlk sürümde daha az özellik ama daha sağlam semantics tercih edilmelidir. C++ gibi her şeyin mümkün olduğu bir dil değil; mümkün olanların güvenli, hızlı ve tahmin edilebilir olduğu bir dil olmalıdır.

Aşama	Ürün hedefi
v0 hedefi	Tek binary compiler + interpreter; HTTP hello world; JSON; Result/Option; arena; SQLite/Postgres adapter taslağı.
v1 hedefi	Production HTTP servisleri, worker pool, package manager, structured diagnostics, test runner.
v2 hedefi	Optimizing native backend, incremental compile, sandbox policy, async IO, FFI stabilization.
v3 hedefi	AI IDE loop, auto-fix diagnostics, formalized spec, larger ecosystem.

5. Syntax ve yüzey tasarımı

Syntax tasarımında amaç en kısa kod değil, en az yanlış anlaşılabilir koddur. İnsan gözü ve AI modeli aynı anda düşünülmelidir. Aşırı sembolik, Perl benzeri kısa ifadeler token açısından cazip görünebilir fakat modelin yanlış üretim riskini artırır. Aşırı verbose Java benzeri syntax ise token tüketimini artırır. Arelang ortada durmalıdır.

Önerilen syntax Go, Zig ve Rust etkisi taşır; fakat Rust kadar çok punctuation ve lifetime syntax içermez. Scope için braces kullanılır. Formatter zorunludur. Noktalı virgül kullanılmaz. Fonksiyon imzaları açık yazılır. Local değişkenlerde type inference vardır ama public API imzalarında tip zorunludur.

```
fn create_user(req: Request) -> Response!ApiError {
    let input = req.json<UserInput>()?
    let user = db.users.insert(input)?
    return Response.json(user)
}

struct UserInput {
    email: Email
    name: String
}

struct User {
    id: UserId
    email: Email
    name: String
}
```

Keyword	Görev	Not
fn	Fonksiyon tanımı	func yerine kısa ama açık.
let	Immutable değişken	Varsayılan güvenlik.
mut	Mutable değişken	Mutasyon görünür.
struct	Kayıt tipi	Backend modelleri için temel.
enum	Tagged union	State ve error modelleri için.
use	Import	Az ceremony.
test	Test bloğu	Dilin birinci sınıf parçası.
unsafe	Güvensiz boundary	Policy ile izlenir.

5.1 Kod stili

Tüm proje tek formatter tarafından biçimlendirilmelidir. Formatter sadece estetik araç değildir; dilin parçasıdır. AI modelleri farklı indentation, import sırası ve satır kırılımı üretirse diff kalitesi düşer ve patch uygulama zorlaşır. Bu nedenle arefmt çıktısı kanonik kabul edilir.

Yorumlar serbesttir fakat generated code için özel doc comment formatı olmalıdır. Public fonksiyonlarda kısa açıklama, hata tipi ve security boundary notu istenebilir. Linter, eksik doc comment yüzünden build kırmamalı; fakat public API stabilizasyonu aşamasında uyarı vermelidir.

6. Type system

Type system static, nominal ve küçük olmalıdır. Structural typing bazı alanlarda ergonomik olsa da backend API'lerinde yanlış tip eşleşmelerine yol açabilir. UserId ile OrderId aynı u64 olsa bile birbirine karışmamalıdır. Bu nedenle opaque type dilin temel yeteneği olmalıdır.

Dil boolean, integer, float, string view, array, slice, struct, enum, option, result, pointer ve function pointer gibi sınırlı fakat yeterli temel yapılarla başlamalıdır. Class inheritance, multiple inheritance, dynamic reflection, unrestricted templates ve implicit interfaces kapsam dışı kalmalıdır.

```
type UserId = opaque u64
type OrderId = opaque u64

struct User {
    id: UserId
    email: Email
}

enum PaymentStatus {
    Pending
    Paid(receipt: ReceiptId)
    Failed(reason: String)
}
```

Tip	Anlam	Tasarım notu
bool	true/false	Numeric cast yok.
i8/i16/i32/i64	Signed integer	Overflow policy build mode'a bağlı.
u8/u16/u32/u64	Unsigned integer	Backend protocol ve binary IO için gerekli.
f32/f64	Float	f16 v0 için kapsam dışı.
String	Owned string	Heap veya arena backed.
Str	String view	Allocation yapmaz.
T?	Optional	Null yerine.
T!E	Result	Exception yerine.
[]T	Slice	Runtime length + pointer.
[N]T	Array	Compile-time length.

6.1 Type inference sınırı

Local değişkenlerde type inference kullanılabilir. Public fonksiyon imzalarında, struct fieldlarında, enum payloadlarında ve FFI boundary üzerinde tip zorunlu olmalıdır. Bu kural hem dokümantasyon hem compiler hızı hem de AI üretimi için önemlidir.

Generic fonksiyonlarda tip parametreleri mümkün olduğunca infer edilebilir olmalı; fakat constraint açık yazılmalıdır. Constraint sistemi küçük tutulmalı ve error mesajları "bu tip şu protokolü sağlamıyor" seviyesinde okunabilir olmalıdır.

```
protocol JsonEncode<T> {
    fn encode(value: T, out: mut JsonWriter) -> Unit!JsonError
}

fn write_json<T: JsonEncode>(res: mut Response, value: T) -> Unit!HttpError {
    return json.encode(value, res.body)?
}
```

7. Memory modeli

Memory modeli dilin en kritik kararıdır. C++ performansı isteniyor fakat C++ gibi manuel lifecycle hataları istenmiyor. Rust güvenliği isteniyor fakat Rust kadar lifetime annotation istenmiyor. Bu nedenle önerilen model: stack-first, arena-first, owned heap explicit ve unsafe raw pointer izole şeklindedir.

Backend kodunda nesnelerin büyük bölümü kısa ömürlüdür. Request parse edilir, validation yapılır, DB response alınır, JSON üretilir ve request biter. Bu pattern için en doğal bellek modeli request arena'dır. Bir request içinde yapılan transient allocation'lar request sonunda topluca temizlenir. Bu hem hızlıdır hem de AI'nin free unutma riskini azaltır.

Alan	Kullanım	Not
Stack	Küçük local değerler	Default ve en güvenli alan.
Request arena	HTTP request ömrü	JSON parse, temporary string, validation error list.
Job arena	Worker job ömrü	Queue processing ve batch işlem.
Owned heap	Requestten uzun yaşayan veri	Cache, connection pool, global config.
Raw pointer	FFI ve unsafe alan	Sadece unsafe blokta.

```
fn handle(req: Request) -> Response!ApiError uses request_arena {
    let input = req.json<UserInput>()? // request arena içinde parse edilir
    let normalized = normalize(input.email) // transient allocation olabilir
    let user = db.users.insert(normalized)? // owned veya DB-backed değer döner
    return Response.json(user)
}
```

Compiler'ın en önemli görevlerinden biri arena escape hatalarını yakalamaktır. Request arena içinde allocate edilen bir değer global cache'e konulması veya long-lived struct içine pointer olarak taşınması yasaklanmalıdır. Bu, Rust lifetime sisteminin daha dar ve backend odaklı bir versiyonu gibi düşünülebilir.

Bu modelde geliştirici her referansın lifetime'ını yazmaz. Bunun yerine compiler birkaç büyük yaşam alanını bilir: stack, request, job, process, unsafe. Değerin daha kısa ömürlü alandan daha uzun ömürlü alana sızması hata üretir.

Memory kuralı

Short-lived alandan long-lived alana pointer taşınamaz. Taşımak istiyorsan clone/own et veya hedef alanı açıkça değiştir.

8. Error handling

Exception tabanlı hata yönetimi bu dil için uygun değildir. Exception control flow'u gizler, AI modelinin hata pathlerini unutmasına neden olur ve low-level performans modelini belirsizleştirir. Go tarzı if err != nil modeli açık olsa da token tüketimi ve tekrar miktarı fazladır. Bu nedenle önerilen model Result + ? propagation olmalıdır.

Tip imzası bir fonksiyonun hangi hata ailesini dönebileceğini göstermelidir. Hata tipleri enum olarak tanımlanmalıdır. HTTP boundary'de hata tiplerinin status code'a map edilmesi standart library tarafından desteklenmelidir.

```
enum ApiError {
    InvalidInput(field: Str, reason: Str)
    Unauthorized
    Db(err: DbError)
    External(err: HttpClientError)
}

fn create_user(req: Request) -> Response!ApiError {
    let input = req.json<UserInput>() map JsonResult => ApiError.InvalidInput("body", "invalid
json")?
    let user = db.users.insert(input) map DbError => ApiError.Db(?)
    return Response.json(user)
}
```

Yapı	Görev	Not
T!E	Başarılı T veya hata E	Fonksiyon imzasında görünür.
?	Hata varsa propagasyon	Token tasarrufu.
map A => B	Hata dönüşümü	Boundary temizliği.
try block	Birden fazla hata pathi	v1 sonrası opsiyonel.
panic	Recover edilemez bug	Production API error için kullanılmaz.

9. Backend standard library

Dilin backend odaklı olmasının anlamı, HTTP ve servis üretimi için gerekli bileşenlerin framework arayışı gerektirmeden gelmesidir. Standart library kapsamı çok geniş tutulursa dil şişer; çok dar tutulursa AI sürekli glue code yazar. Bu nedenle stdlib katmanlı tasarlanmalıdır: çekirdek, backend prelude, optional adapters.

Çekirdek dil küçük kalmalı; backend prelude ise opinionated olmalıdır. HTTP server, routing, JSON, validation, logging, metrics ve test araçları dilin resmi paketleri içinde bulunmalıdır. Database, queue ve cloud provider adapterları resmi ama opsiyonel modüller olabilir.

Modül	İçerik
http	Server, router, request, response, middleware, status mapping
json	Zero-copy parse seçenekleri, typed decode/encode, streaming writer

validate	Email, URL, length, pattern, custom rule, schema errors
config	env, file, secret source, typed config loading
log	structured logging, request correlation id
metrics	counter, histogram, gauge, default /metrics endpoint
trace	span, context propagation, sampling
db	query interface, pool, transaction, migration primitive
queue	publish, consume, retry, dead letter
auth	password hash, JWT verify/sign, session primitive, OAuth helper
test	unit, integration, HTTP test client, golden tests
bench	microbenchmark, allocation report, p95 latency helper

```

service App {
  use log.default()
  use metrics.default()
  use trace.request_id()

  route POST "/users" -> create_user
  route GET "/users/:id" -> get_user
  route GET "/health" -> health.ok
}

```

9.1 Validation

Validation backend dillerinde çoğu zaman framework veya reflection ile çözülür. Bu dilde reflection'ın büyümesine izin vermeden schema tabanlı validation sağlanmalıdır. Struct field attribute'ları kontrollü olabilir, fakat macro gibi arbitrary code generation'a dönüşmemelidir.

AI açısından validation'ın standardize olması çok önemlidir. Modelin her endpoint için farklı error shape üretmesi yerine standard library aynı ValidationError yapısını döndürmelidir.

```

struct UserInput {
  email: Email @validate.email
  name: String @validate.len(min=2, max=80)
}

```

10. Concurrency ve worker modeli

Concurrency modeli Go kadar anlaşılır fakat data race açısından daha güvenli olmalıdır. AI modellerine manuel thread, lock, unlock ve condition variable yazdırmak hata oranını artırır. Dil task, channel, worker pool ve shared state kavramlarını standartlaştırmalıdır.

Varsayılan olarak değerler tasklar arasında immutable taşınmalı veya ownership transfer edilmelidir. Paylaşılan mutable state explicit guarded_by veya atomic olarak işaretlenmelidir. Compiler, mut shared state'e korumasız erişimi hata yapmalıdır.

```

worker ImageJobWorker(pool: WorkerPool<ImageJob>) {
  for job in pool.jobs() {
    process_image(job)?
  }
}

```

```

shared Cache<K, V> guarded_by Mutex
atomic request_count: u64

```

Yapı	Görev	Not
spawn	Lightweight task başlatır	Return handle tipli olur.
channel<T>	Typed message passing	Backpressure destekli.
select	Channel/timeout seçimi	Go benzeri ama typed.
worker pool	Backend job processing	Retry ve cancellation ile.
context	Cancellation/deadline	HTTP, DB, queue ile taşınır.

11. Modül ve paket sistemi

Modül sistemi Go kadar düz olmalıdır. Header dosyası yoktur. Circular import yasaktır veya çok sınırlıdır. Paket dosyası tek ve okunabilir olmalıdır. Al'nin build config üretirken hata yapmaması için package manifest minimal tutulmalıdır.

Import isimleri filesystem ile uyumlu olmalı, fakat public API dışı açık olarak export edilmelidir. Wildcard import olmamalıdır. Import alias desteklenebilir ama formatter tarafından normalize edilir.

```
// are.pkg
name = "users-api"
version = "0.1.0"
target = "server"

use http = "std/http@1"
use postgres = "std/db/postgres@1"
use jwt = "std/auth/jwt@1"

use std.http
use std.json
use std.db.postgres as pg
use app.users
```

12. Compiler mimarisi

İlk compiler üretimi için en mantıklı yaklaşım küçük ve test edilebilir parçalara ayrılmış bir mimaridir. Lisyrus makalesindeki parser, AST, IR, interpreter ve JIT ayrımı pratik bir başlangıç noktasıdır. Arelang için de benzer ayrım kullanılmalı fakat backend ve AI diagnostics baştan mimariye eklenmelidir.

Compiler ilk aşamada LLVM backend kullanabilir. Kendi native code generator'ını yazmak öğretici fakat ürünleşme hızını düşürür. Dil semantics'i oturana kadar LLVM, Cranelift veya benzeri bir backend kullanmak daha gerçekçidir. Daha sonra performans veya binary boyutu için özel backend düşünülebilir.

Bileşen	Görev	Not
lexer	Token üretir	Hatalı Unicode, invalid literal, comment handling.
parser	AST üretir	Error recovery ve partial AST önemli.
resolver	İsimleri bağlar	Scope, import, export kontrolü.
type checker	Tipleri doğrular	Option/Result, arena escape, mutability.
borrow/arena checker	Lifetime alanlarını denetler	Request -> global kaçış hatalarını yakalar.
lowering	AST -> HIR/MIR/IR	Backend bağımsız ara temsil.
optimizer	Basit optimizasyonlar	Constant folding, DCE, inline.
codegen	LLVM/Cranelift/native	v0'da LLVM daha pratik.
diagnostics	Hata üretir	JSON ve human-readable output.
formatter/linter	Kanonik stil	AI loop için kritik.

```
source.arena
-> lexer tokens
-> parser AST
-> resolver bound AST
-> type checker typed AST
-> arena checker checked HIR
-> IR
-> optimizer
-> codegen
-> binary / wasm / object / test runner
```

12.1 Compiler implementasyon dili

Compiler'ın ilk sürümü için Rust, Go veya C++ düşünülebilir. Rust güvenlik ve compiler yazma kültürü açısından güçlüdür; Go hızlı geliştirme ve basitlik açısından avantajlıdır; C++ ise LLVM ekosistemi ve low-level kontrol açısından doğaldır fakat hata yüzeyi daha büyüktür. Bu proje için en makul seçenek Rust veya Go'dur.

Eğer hedef kısa sürede çalışan bir MVP ise Go ile lexer/parser/type checker yazmak daha hızlı olabilir. Eğer hedef uzun vadeli sağlam compiler ise Rust daha güvenli olabilir. LLVM entegrasyonu gerekiyorsa Rust tarafında inkwell gibi wrapperlar veya C API üzerinden entegrasyon düşünülebilir. v0 için interpreter + IR + C backend bile yeterli olabilir: önce semantics, sonra performans.

Dil	Avantaj	Dezavantaj
Go	Hızlı geliştirme, sade kod, kolay tooling	Compiler içi memory safety Rust kadar güçlü değil.
Rust	Güvenli compiler, güçlü enum/pattern matching	Geliştirme hızı daha yavaş olabilir.
C++	LLVM ve native kontrol	Karmaşıklık ve memory bug riski.
OCaml	Compiler geleneği	Ekip/ekosistem bariyeri.

13. IR ve optimizasyonlar

IR tasarımında ilk hedef en iyi optimizer değil, semantics'i net temsil eden ve hata ayıklanabilir bir yapı olmalıdır. AST çok yüzeysel syntax'a bağlıdır; IR ise compiler'ın gerçek çalışma dilidir. İyi IR, hem interpreter hem native backend hem static analysis için ortak zemin sağlar.

v0 için SSA benzeri fakat basit bir typed IR yeterlidir. Her node tip taşınalıdır. Control flow basic blocklar ile temsil edilmelidir. Struct field access, array/slice access, function call, branch, return, alloc, arena_alloc, load/store ve error propagation ayrı node türleri olarak bulunmalıdır.

IR node	Görev
const	Literal değer
alloc_stack	Stack allocation
alloc_arena	Arena allocation
load/store	Memory erişimi
field/get/set	Struct field işlemi
call	Fonksiyon çağırısı
call_foreign	FFI çağırısı
branch	Conditional jump
return	Fonksiyon çıkışı
result_check	? propagasyon lowering
arena_escape_check	Analysis node veya pass işareti

İlk optimizasyon seti sınırlı olmalıdır: constant folding, dead code elimination, simple inlining, bounds check elimination, escape analysis ve allocation sinking. LLVM kullanılıyorsa bu optimizasyonların bir kısmı backend tarafından yapılır; fakat arena escape ve Result/Option lowering gibi dil spesifik analizler ön tarafta kalmalıdır.

14. AI-agent uyumluluğu

Bu dilin ayırt edici tarafı AI-agent uyumluluğunu sonradan eklenen bir araç değil, dilin temel tasarım kriteri olarak ele almasıdır. AI kod üretirken hata yapacaktır; önemli olan bu hataların compiler tarafından erken, açık ve düzeltilebilir şekilde yakalanmasıdır.

Compiler çıktısı iki formatta olmalıdır: insan için okunabilir metin ve AI için JSON diagnostics. JSON diagnostic, error code, file, range, problem, reason, fix suggestions ve safety level içermelidir. IDE ve agent bu veriyi kullanarak otomatik patch önerebilir.

```
{
  "code": "E_Arena_1042",
  "severity": "error",
  "file": "src/users.arena",
  "range": { "start": [42, 17], "end": [42, 31] },
  "problem": "value allocated in request arena escapes into process cache",
  "reason": "UserProfile is tied to request lifetime but Cache requires process lifetime",
  "fixes": [
    { "label": "clone into process heap", "edit": "profile.clone_to(process_heap)" },
    { "label": "store only UserId", "edit": "cache.set(user.id)" }
  ]
}
```

Özellik	AI katkısı
Deterministik formatting	Patch daha güvenli uygulanır.
Küçük grammar	Model daha az pattern karıştırır.

Structured diagnostics	Agent kendi hatasını düzeltebilir.
Canonical project layout	Dosya organizasyonu tahmin edilebilir.
Stdlib backend primitives	Glue code azalır.
No hidden control flow	Model execution path'i daha iyi izler.
Policy-aware unsafe	Agent'a "unsafe kullanma" kuralı compiler ile uygulanır.

15. Güvenlik ve sandbox

Sandbox dilin opsiyonel eklentisi değil, backend ve AI-generated code için temel güvenlik aracıdır. AI'nin ürettiği kod her zaman güvenilir kabul edilemez. Bir servis generator, plugin runner veya workflow system içinde üretilen kodun filesystem, network, process, environment ve secret erişimleri capability ile kontrol edilmelidir.

Her program veya modül capability manifest ile derlenmelidir. Varsayılan olarak network, filesystem, process spawn ve env secret erişimi kapalıdır. İzinler route, module veya binary seviyesinde açılır. FFI çağrıları da policy kontrolünden geçmelidir.

```
capabilities {
  network.outbound = ["api.stripe.com", "api.openai.com"]
  filesystem.read = ["/app/config"]
  filesystem.write = ["/tmp/app"]
  env.read = ["DATABASE_URL", "STRIPE_SECRET"]
  process.spawn = false
}
```

Alan	Kontrol
Filesystem	read/write path allowlist
Network	host/port/protocol allowlist
Env/secrets	named secret access
Process	spawn/exec default deny
FFI	declared foreign functions only
Unsafe	module-level report + policy gate
Resource limits	CPU, memory, timeout, open fd

16. Örnek backend uygulaması

Aşağıdaki örnek, dilin hedef ergonomisini gösterir. Amaç gerçek compiler syntax'ının keskinleşmiş hali değildir; tasarım yönünü somutlaştırmaktır. Endpoint kısa görünür çünkü routing, validation, JSON, error mapping ve tracing standard library tarafından taşınır.

```
use std.http
use std.json
use std.validate
use std.db.postgres as pg
use std.auth.jwt

struct AppState {
  db: pg.Pool
  jwt: jwt.Verifier
}

struct CreateUserInput {
  email: Email @validate.email
  name: String @validate.len(min=2, max=80)
}

struct UserDto {
  id: UserId
  email: Email
  name: String
}

enum ApiError {
```

```

    InvalidInput(err: ValidationError)
    Unauthorized
    Db(err: pg.Error)
}

fn create_user(ctx: RequestContext<AppState>, req: Request) -> Response!ApiError uses
request_arena {
    let claims = ctx.jwt.require(req.headers.authorization)? map AuthError =>
ApiError.Unauthorized
    let input = req.json<CreateUserInput>()? validate input map ValidationError =>
ApiError.InvalidInput(?)

    let user = ctx.state.db.query_one<UserDto>(
        "insert into users(email, name, owner_id) values ($1, $2, $3) returning id, email, name",
        input.email,
        input.name,
        claims.user_id
    )? map pg.Error => ApiError.Db(?)

    return Response.created(user)
}

service UsersApi(state: AppState) {
    use http.trace()
    use http.metrics()
    use http.error_map<ApiError>()

    route POST "/users" -> create_user
    route GET "/health" -> http.health.ok
}

```

Bu örnekte birkaç karar bilinçlidir. Request arena explicit olarak görünür fakat her allocation tek tek yazılmaz. Hata dönüşümü görünürdür fakat Go'daki gibi her satırda if err != nil yoktur. Validation, JSON ve HTTP mapping standarttır. Auth boundary görünürdür. SQL string örnek olarak verilmiştir; v1'de query builder veya prepared query macro'su değil, compiler-checked query interface tercih edilebilir.

17. MVP geliştirme yol haritası

Dil geliştirme projesi kolayca sonsuz bir araştırma projesine dönüşebilir. Bu nedenle MVP çok dar tutulmalıdır. İlk hedef “her şeyi yapan mükemmel dil” değil, “küçük bir backend servisini derleyip çalıştıran, AI'nin kod üretebildiği, hataları iyi raporlayan dil” olmalıdır.

Yol haritası iki paralel hatla ilerlemelidir: dil semantics hattı ve backend ürün hattı. Dil hattı lexer, parser, type checker, IR, interpreter/codegen tarafını ilerletir. Ürün hattı ise HTTP, JSON, validation, test runner ve diagnostics deneyimini geliştirir.

Dönem	Çıktı	Kapsam
Hafta 1-2	Spec v0	Keywordler, type grammar, error model, memory scope kuralları.
Hafta 3-4	Lexer/parser	AST üretimi, formatter iskeleti, syntax testleri.
Hafta 5-6	Resolver/type checker	Struct, enum, function, Result, Option, mutability.
Hafta 7-8	Interpreter	AST veya IR interpreter ile hello world, arithmetic, function call.
Hafta 9-10	IR	Typed IR, control flow, function lowering, error propagation.
Hafta 11-12	HTTP MVP	Minimal server, route, JSON decode/encode.
Hafta 13-14	Arena model	Request arena ve escape checker v0.
Hafta 15-16	Diagnostics	Human + JSON output, auto-fix suggestion skeleton.
Hafta 17-20	Native/codegen	LLVM/Cranelft/C backend; benchmark harness.
Hafta 21-24	Dogfood	Bir gerçek internal API'nin bu dille yazılması.

17.1 İlk repository yapısı

```
arelang/  
  compiler/  
    lexer/  
    parser/  
    ast/  
    resolver/  
    typecheck/  
    arena_check/  
    ir/  
    codegen/  
    diagnostics/  
  std/  
    core/  
    http/  
    json/  
    validate/  
    log/  
    metrics/  
    db/  
  tools/  
    arefmt/  
    aretest/  
    arepkg/  
  examples/  
    hello_http/  
    users_api/  
    worker_queue/  
  spec/  
    language.md  
    memory.md  
    diagnostics.md  
    stdlib.md
```

18. Test stratejisi

Dil projesinde test yalnızca runtime davranışını test etmek değildir. Lexer snapshot, parser golden AST, type checker error, formatter idempotence, diagnostics JSON schema, IR lowering, codegen correctness, benchmark regression ve sandbox policy testleri ayrı ayrı kurulmalıdır.

AI odaklı bir dilde ayrıca “model generated corpus” tutulmalıdır. Farklı LLM'lere aynı backend görevleri verilir, ürettikleri kodlar compiler'dan geçirilir, hata türleri sınıflandırılır. Dil tasarımı bu veriye göre sadeleştirilir. Bu, klasik dil geliştirme süreçlerinde pek yapılmayan fakat bu proje için kritik bir ölçümdür.

Test türü	Kontrol
Lexer tests	Token stream beklenenle aynı mı?
Parser tests	AST snapshot stabil mi?
Type tests	Doğru program geçiyor, yanlış program doğru hata veriyor mu?
Formatter tests	Format(format(code)) == format(code) mi?
Diagnostics tests	JSON schema ve fix suggestion doğru mu?
IR tests	Lowering expected IR üretiyor mu?
Runtime tests	Interpreter/native aynı sonucu veriyor mu?
Fuzz tests	Parser ve type checker crash etmiyor mu?
AI corpus tests	LLM görevleri kaç düzeltmede derleniyor?
Benchmark tests	Latency, allocation ve binary size regresyonu var mı?

19. Kaçınılacak özellikler

Yeni dilin başarısı hangi özellikleri eklediğinden çok, hangi özellikleri bilinçli olarak dışarıda bıraktığıyla belirlenecektir. C++ tarihsel olarak çok güçlüdür fakat tam da bu yüzden AI ve yeni başlayan insanlar için risklidir. Arelang'ın ilk sürümü "hayır" diyebilen bir dil olmalıdır.

Aşağıdaki özellikler ya tamamen dışarıda bırakılmalı ya da çok daha sonraki sürümlere ertelenmelidir. Her biri ergonomi sunabilir; fakat compiler karmaşıklığı, AI hata oranı, token tüketimi veya güvenlik riski doğurur.

Kaçınılacak şey	Neden
Implicit cast	Gizli veri kaybı ve yanlış overload.
Null pointer	Eksik check ve runtime crash.
Exception	Gizli control flow.
Text macro	Parser/diagnostic karmaşası, güvenlik riski.
Multiple inheritance	Model ve insan için semantik karmaşa.
Unrestricted templates	C++ benzeri cryptic errors.
Operator overloading	Gizli davranış; en fazla sınırlı numeric/vector domain.
Header files	Build ve dependency karmaşası.
Global mutable default	Race condition ve test zorluğu.
Reflection-heavy framework	Runtime overhead ve magic.
GC zorunluluğu	Latency belirsizliği; opsiyonel managed heap v2 olabilir.
Arbitrary compile-time execution	Build güvenliği ve determinism riski.

20. Açık kararlar ve gelecek işler

Bazı kararlar bilinçli olarak açık bırakılmalıdır. Dil tasarımında erken kesinlik bazen yanlış kilitlenme yaratır. Ancak açık kararların her biri ölçülebilir deneylerle kapatılmalıdır. Örneğin brace vs indentation kararı ideolojik değil; formatter, patch stability ve AI corpus sonuçlarıyla değerlendirilmelidir.

Özellikle backend standard library kapsamı dikkatli yönetilmelidir. Çok fazla şeyi stdlib'e almak dilin bakım yükünü artırır; çok az şeyi almak token ekonomisini bozar. Burada resmi paketler ve çekirdek stdlib ayrımı yardımcı olur.

Karar	Soru	Önerilen çözüm yöntemi
Compiler implementation language	Rust mı Go mu?	MVP hızı ve uzun vadeli güvenlik kıyaslanmalı.
Backend codegen	LLVM/Cranelift/C backend?	İlk ürün için en hızlı çalışan seçenek seçilmeli.
Async model	Go-like task mı async/await mi?	Backend ergonomisi ve compiler karmaşıklığı ölçülmeli.
Generics	Protocol sistemi ne kadar güçlü?	Error mesajı kalitesi öncelikli olmalı.
Database API	SQL string, query builder, typed query?	AI hata oranı ve developer ergonomisi test edilmeli.
Arena syntax	uses request_arena gerekli mi?	Görünürlük ile token ekonomisi dengelenmeli.
Sandbox runtime	Native policy mi WASM target mı?	İlk wedge'e göre karar verilmeli.

21. Kaynak notları

Bu tasarım dokümanı, önceki analizlerde yapılan AI kod üretimi, low-level performans, Zig/Lua/C++/Rust/Go kıyasları ve Lisyarus'un "Making your own programming language is easier than you think (but also harder)" yazısındaki pratik compiler/dil tasarımı bulguları üzerine kuruludur.

Lisyarus yazısında pslang adlı küçük bir dilin tasarım hedefleri anlatılır: seamless C interop, low-level çalışma, sandbox, küçük compiler footprint, hızlı compilation, cross-platform hedefler ve C++'ı yeniden yaratmamaya çalışma. Yazıda ayrıca static/strict/nominal type system, no implicit casts, C ABI fonksiyonlar, parser-AST-IR-interpreter/JIT ayrımı, IR optimizasyon planları ve otomatik resource management seçenekleri ele alınır.

Bu dokümanda o bulgular backend ve AI-native geliştirme odağıyla revize edilmiştir. En önemli ayrışmalar null yerine Option, manual alloc/free yerine arena-first memory, indentation yerine formatterlı brace syntax, C++ tarzı unrestricted templates yerine sınırlı protocol/generic sistemi ve compiler diagnostics'in machine-readable tasarlanmasıdır.

Kaynak	İçerik	Not
Lisyarus blog	Making your own programming language is easier than you think (but also harder)	https://lisyarus.github.io/blog/posts/making-your-own-programming-language.html

Tasarım yakınlığı	Zig, Go, Rust, pslang, C ABI yaklaşımı	Dil ailesi kıyasları ve önceki analiz bulguları
Ana hipotez	AI için iyi dil = küçük grammar + açık semantics + güçlü stdlib + structured diagnostics	Bu dokümanın merkezi varsayımı

Ek A - Mini dil spesifikasyonu taslağı

Bu ek, Arelang v0 için tek sayfalık spec gibi okunabilir. Detaylı grammar daha sonra ayrı dosyaya çıkarılmalıdır. Buradaki amaç compiler geliştirmeye başlanmadan önce kararları sabitlemektir.

Konu	Karar
File extension	.are
Formatter	arefmt zorunlu
Package manifest	are.pkg
Entry	service veya fn main()
Scope	Brace-based
Statement separator	Newline; semicolon yok
Comment	// line, /* block */
Public API	pub keyword
Error	!E Result shorthand
Option	T?
Unsafe	unsafe fn/block/module
Allocation	stack default, arena explicit, heap explicit
FFI	foreign fn + capability policy

Ek B - İlk 10 örnek program

Örnek	Amaç
Hello HTTP	Minimal service ve health check.
JSON decode	Typed request body parse.
Validation error	Invalid input response.
Database insert	Postgres query.
Worker	Queue consume ve retry.
Auth	JWT verify.
Metrics	Counter/histogram.
Arena escape error	Compiler hata örneği.
FFI call	C function boundary.
Unsafe report	Unsafe kullanım raporu.

Ek C - Örnek compiler diagnostic seti

Kod	Ad	Açıklama
E_TYPE_0001	Type mismatch	Beklenen ve gelen tip farklı.
E_NULL_0002	Null forbidden	Null literal yok; Option kullan.
E_ARENA_1042	Arena escape	Kısa ömürlü değer uzun ömürlü alana taşındı.
E_MUT_0201	Mutation forbidden	Immutable değer değiştirilmeye çalışıldı.
E_UNSAFE_0301	Unsafe not allowed	Policy unsafe kullanımı engelledi.
E_CAP_0401	Capability denied	Network/filesystem/env izni yok.
E_RESULT_0501	Unhandled result	!E sonucu handle edilmedi.
E_OPTION_0601	Unchecked option	T? unwrap edilmeden kullanıldı.
E_IMPORT_0701	Import cycle	Modül döngüsü var.
E_FFI_0801	Foreign boundary mismatch	C ABI tipi uyumsuz.

Sonuç

Backend odaklı küçük bir dil yaratmak ancak hedef dar tutulursa rasyonel bir projedir. Dil, C++ veya Rust gibi geniş ekosistemlerle doğrudan rekabet etmeye çalışmamalıdır. Bunun yerine AI tarafından üretilen, güvenli defaultlara sahip, native performansa yakın, backend standart işleri güçlü şekilde kapsayan bir niş açmalıdır.

Bu dokümanın önerdiği çizgi, pratik olarak Go ergonomisi ile Zig açıklığını birleştirip Rust güvenlik sezgisini daha dar ve backend odaklı bir modele indirger. En kritik yenilik ise AI-agent uyumluluğunun compiler diagnostics, syntax, stdlib ve memory model seviyesinde düşünülmesidir.

İlk uygulanabilir hedef şudur: AI'ye “auth + validation + DB + JSON + metrics içeren küçük bir backend yaz” dendiğinde, modelin ürettiği kodun kısa, okunabilir, güvenli ve bir iki compiler düzeltmesiyle production'a yaklaşabilir olması. Bu hedef gerçekçi, ölçülebilir ve yeterince dar bir başlangıç noktasıdır.

Derin Teknik Ekler

Bu bölüm, önceki ana tasarım kararlarını uygulanabilir compiler ve standard library görevlerine indirger. Amaç yalnızca dilin nasıl görüneceğini anlatmak değil, bu dili gerçekten geliştirmek isteyen bir ekibin hangi dosyaları, hangi passleri, hangi testleri ve hangi sınır kararlarını oluşturacağını göstermektir.

Ek D - Lexical grammar ve token seti

Lexer basit olmalıdır. Dilde context-sensitive token davranışı minimumda tutulmalıdır. Unicode identifier desteği v1 sonrasına bırakılabilir; v0 için ASCII identifier tercih edilmelidir. Bunun sebebi kültürel kapsayıcılığı reddetmek değil, compiler MVP'sini ve AI üretimini deterministik tutmaktır.

Token	Biçim	Karar
Identifier	[A-Za-z_][A-Za-z0-9_]*	v0 için ASCII.
Integer literal	123, 123u, 123i64	Suffix açık ve sınırlı.
Float literal	1.0, 1.0f64	Noktanın iki tarafı zorunlu.
String literal	"text"	UTF-8; escape kuralları sınırlı.
Line comment	// comment	Formatter korur.
Block comment	/* comment */	Nested block comment v0'da yok.
Operator	+, -, *, /, %, ==, !=, <, <=, >, >=	Operator overloading yok.
Punctuation	{ } () [] , . : -> ! ?	Az ve düzenli.

```
// Keyword set v0
fn let mut struct enum type opaque protocol impl
if else while for in return break continue
use pub test unsafe foreign service route worker
true false
```

Keyword sayısı sınırlı tutulmalıdır. Yeni keyword eklemek geriye dönük uyumluluk ve parser karmaşıklığı doğurur. Bu yüzden framework benzeri kavramlar keyword yerine stdlib API olarak başlamalı, yalnızca dil semantics'i gerektiriyorsa keyword'e dönüşmelidir.

Ek E - Parser ve grammar yaklaşımı

Parser için recursive descent veya Pratt parser yaklaşımı v0'da yeterlidir. Bison/Yacc benzeri araçlar güçlüdür fakat hata recovery ve IDE entegrasyonu için elle yazılmış parser daha esnek olabilir. Backend ve AI odaklı dilde parse hatası yalnızca "syntax error" dememeli; yanlış brace, eksik tip, eksik Result handling gibi sık üretim hatalarını açıkça sınıflandırmalıdır.

```
file      = item* EOF
item      = use_decl | struct_decl | enum_decl | type_decl | fn_decl | service_decl | test_decl
fn_decl   = visibility? "fn" ident "(" params? ")" return_type? effect_clause? block
return_type = "->" type
result_type = type "!" type
option_type = type "?"
block     = "{" stmt* "}"
stmt      = let_stmt | assign_stmt | if_stmt | while_stmt | for_stmt | return_stmt | expr_stmt
expr      = pratt_expression
```

Grammar özel olarak tek bir okunabilir stile zorlanmalıdır. Birden fazla eşdeğer yazım varsa LLM aynı işi farklı yazabilir ve corpus kararsızlaşır. Örneğin function declaration için hem fn hem func desteklemek kısa vadede hoş görünür fakat uzun vadede gereksiz varyasyon üretir.

Ek F - Semantic pass sırası

Compiler passleri birbirinden ayrı tutulmalıdır. Her pass küçük, test edilebilir ve kendi error kodlarını üretmelidir. Bu ayrım hem insan geliştiricinin compiler'a katkı vermesini kolaylaştırır hem de AI-agent'in compiler içindeki bir bug'u lokalize edebilmesini sağlar.

Sıra	Pass	Görev
1	Parse	Syntax ağacı ve recoverable hata listesi.
2	Name resolution	Identifier -> declaration bağlama.
3	Import resolution	Paket ve modül grafi oluşturma.
4	Type inference/check	Expression tiplerini ve public API imzalarını

		doğrulama.
5	Effect/capability check	Network, file, env, unsafe izinlerini doğrulama.
6	Arena/lifetime check	Scope escape ve ownership hatalarını yakalama.
7	Result/Option exhaustiveness	Unhandled error/option durumlarını yakalama.
8	Lowering to HIR	Syntax şeker temizleme.
9	Lowering to IR	Control flow ve memory operation temsil etme.
10	Codegen/test/interpreter	Çalıştırma veya binary üretme.

Pass sırası tasarımın kalbidir. Capability check type checking'den önce yapılırsa tip bilgisi eksik kalır; arena check IR sonrasında bırakılırsa hata mesajı kaynak koda bağlanmakta zorlanır. Bu yüzden HIR seviyesi, compiler diagnostics için merkezi ara temsil olmalıdır.

Ek G - Memory checker algoritması

Arena ve ownership sistemi için tam Rust benzeri borrow checker yazmak gerekmez. Daha dar bir backend problemi çözülebilir. Her allocation bir region etiketi alır: stack, request, job, process veya unsafe. Her referans ve pointer bu region etiketiyle taşınır. Assignment, return, struct field set ve function call sırasında region compatibility kontrol edilir.

```
region_order:
    stack <= request <= job <= process

rule escape:
    value.region must live at least as long as target.region

examples:
    request -> process : error unless clone_to(process)
    process -> request : ok
    stack -> return    : error if returned by reference
    request -> response : ok if response serialized before request end
```

Bu model tüm memory safety problemlerini çözmez. Ancak backend kodunda en yaygın ve AI için en tehlikeli hata sınıfını kapatır: kısa ömürlü veriyi uzun ömürlü state'e sızdırmak. Raw pointer ve FFI için ayrıca unsafe boundary gerekir.

Hata	Açıklama	Diagnostic
Return reference to local	Stack referansı fonksiyondan çıkıyor	E_REGION_RETURN_LOCAL
Cache request object	Request arena verisi process cache'e yazılıyor	E_ARENA_ESCAPE
Async task captures request	Spawn edilen task request bitince çalışabilir	E_TASK_CAPTURE
FFI pointer stored	Foreign pointer ownership belirsiz	E_FFI_OWNERSHIP
Slice outlives buffer	Slice'in dayandığı buffer daha kısa ömürlü	E_SLICE_REGION

Ek H - Backend runtime ve deployment modeli

Dil runtime'ı küçük olmalıdır. GC, büyük event loop runtime'ı veya reflection metadata zorunlu olmamalıdır. Fakat backend için bazı runtime parçaları gereklidir: scheduler, timer, network poller, metrics registry, panic handler, allocator ve signal handling. Bu parçalar modüler tutulmalı; CLI veya embedded plugin hedeflerinde devre dışı bırakılabilmelidir.

Target	Runtime bileşenleri	Kullanım
server target	HTTP server, scheduler, network poller, metrics	Varsayılan backend hedefi.
worker target	Queue consumer, retry runtime, signal handling	Background jobs.
cli target	Arg parser, filesystem, stdout/stderr	Küçük binary.
plugin target	Sandbox, capability manifest, host ABI	Stage benzeri generated plugins.
library target	C ABI export, no main runtime	Host uygulamaya gömülür.

Deployment çıktısı tek binary olmalıdır. Container içinde çalışabilir ama container zorunlu olmamalıdır. Binary kendi health endpointini, build metadata'sını ve version bilgisini standart şekilde sunmalıdır. Bu, generated backendlerin operasyonel kalitesini artırır.

Ek I - HTTP framework değil, dil standardı

HTTP desteği framework gibi değil, resmi backend standardı gibi düşünülmelidir. Bu, her şeyi keyword yapmak anlamına gelmez. service ve route syntax'ı dilin parçası olabilir; middleware ve adapterlar stdlib API olabilir. Burada amaç kullanıcıyı framework seçimi, dependency sürümü ve plugin uyumsuzluğu ile boğmamaktır.

```
service BillingApi(state: AppState) {
  use http.timeout(5s)
  use http.body_limit(1mb)
  use auth.require_bearer(state.jwt)
  use metrics.route_latency()

  route POST "/checkout" -> create_checkout
  route POST "/webhooks/stripe" -> stripe_webhook without auth.require_bearer
}
```

Route tanımı declarative olmalıdır. Bu, compiler'ın route listesi, OpenAPI şeması, test client, permission audit ve missing handler kontrollerini üretmesini sağlar. AI'nin route stringleriyle handlerları yanlış bağlama riski azalır.

Özellik	Fayda
Route registry	Compile-time route listesi çıkarılır.
OpenAPI export	Struct ve validation'dan schema üretilir.
Typed params	/users/:id UserId olarak parse edilir.
Error mapping	ApiError -> HTTP status central yapılır.
Middleware audit	Her route hangi middleware'den geçiyor görülebilir.

Ek J - Database tasarım kararı

Database API, dilin en riskli alanlarından biridir. Runtime SQL stringleri esnek ama hataya açıktır. Compile-time SQL checking güçlü ama database bağlantısı, migration state ve build determinism problemleri yaratır. v0 için en iyi çözüm typed query interface + optional offline schema snapshot olabilir.

```
// schema snapshot: db/schema.are.json
let user = db.query_one<UserDto>(
  sql "select id, email, name from users where id = $1",
  user_id
)?
```

sql literal özel bir string literal olabilir; ancak bu macro sistemine dönüşmemelidir. Compiler, offline schema snapshot varsa column ve parameter sayısını kontrol eder. Snapshot yoksa runtime prepared query kullanılır ve compiler uyarı verir. Bu yaklaşım AI'ye güvenli default sunar ama geliştirme akışını kırmaz.

Yaklaşım	Avantaj	Risk
Raw SQL string	Kolay ve güçlü	AI kolon/parametre hatası yapabilir.
Query builder	Typed olabilir	Verbose ve token maliyeti yüksek.
Compile-time SQL	Güvenli	Build ortamı karmaşık.
Schema snapshot	Dengeli	Snapshot güncelliği yönetilmeli.

Ek K - Auth ve secret yönetimi

Auth stdlib içinde primitive olarak bulunmalıdır fakat policy açık olmalıdır. Password hashing, JWT verify/sign, session cookie, CSRF, OAuth callback ve webhook signature verification resmi modüller olarak gelmelidir. AI modelleri kriptografik ayrıntıları kendisi yazmamalıdır.

Güvenlik kuralı

AI-generated kodun kendi password hashing algoritması, token formatı veya signature verification mantığı yazmasına izin verilmemelidir. Bunlar stdlib fonksiyonlarıyla yapılmalıdır.

```
let password_hash = auth.password.hash(input.password)?
let ok = auth.password.verify(input.password, user.password_hash)?
let claims = auth.jwt.verify<UserClaims>(token, state.jwt_key)?
```

```
let valid = auth.webhook.stripe.verify(req, state.stripe_webhook_secret)?
```

Secret erişimi capability sisteminden geçmelidir. Kod doğrudan environment okuyabilir gibi görünmemelidir. Secret, typed config loader veya secret provider ile alınmalıdır. Bu sayede generated code'un bütün environment'ı dump etmesi veya yanlış secret kullanması engellenebilir.

Ek L - Observability by default

Generated backendlerin en büyük zayıflıklarından biri çalıştıktan sonra görünmez olmalarıdır. Dil standard library'si observability'yi başlangıçtan sağlamalıdır. Log, metrics, trace ve health check elle eklenen opsiyonel süsler değil, service target'ın doğal parçası olmalıdır.

Alan	Varsayılan
Logging	JSON structured log, request_id, span_id
Metrics	Route latency, error count, request count, queue lag
Tracing	Context propagation, db/http span
Health	/health, /ready, dependency status
Profiling	Allocation report, pprof benzeri endpoint optional
Audit	Capability use, unsafe use, secret access log

Bu karar token ekonomisine de hizmet eder. AI'ye her projede logging middleware yazdırmak yerine service declaration bunu otomatik bağlar. Kullanıcı isterse kapatabilir ama default production-friendly olmalıdır.

Ek M - İnsan için öğrenme yolu

Dil yalnızca AI için tasarlanırsa insanlar tarafından sahiplenilmez. İnsan öğrenme yolu doğal olmalıdır. İlk gün syntax ve HTTP hello world; ilk hafta database, validation, testing; ilk ay memory model, FFI ve compiler diagnostics anlaşılmalıdır.

Süre	Öğrenilecek alan
1. gün	Syntax, let/mut, fn, struct, Result, Option, HTTP hello world
1. hafta	JSON, validation, DB, tests, package manifest
1. ay	Arena, worker, capability, FFI, profiling
3. ay	Compiler passleri, stdlib katkısı, native codegen temelleri

Dokümantasyon "dil teorisi" ile başlamamalıdır. İlk sayfa çalışan servis göstermelidir. Ardından hata yönetimi, validation ve memory modeli örneklerle açıklanmalıdır. Compiler içi konular ayrı advanced bölümde tutulmalıdır.

Ek N - AI değerlendirme benchmarkı

Bu dilin başarısı klasik benchmarklarla ölçülemez. C++'tan hızlı mı, Go'dan kısa mı gibi sorular önemlidir; fakat ana iddia AI üretilebilirliğidir. Bu yüzden özel bir benchmark seti gerekir.

Benchmark	Görev	Ölçüm
Task 1	CRUD API + validation + DB insert	İlk derleme oranı, token sayısı.
Task 2	Stripe webhook verify	Security primitive doğru kullanıldı mı?
Task 3	Queue worker + retry	Concurrency ve error handling doğru mu?
Task 4	Arena escape trap	Compiler hatayı yakalıyor mu, AI düzeltiyor mu?
Task 5	Unsafe forbidden policy	Model unsafe kullanınca build duruyor mu?
Task 6	OpenAPI export	Route ve schema doğru mu?
Task 7	Migration mismatch	DB schema hata mesajı anlaşılır mı?
Task 8	Timeout/cancellation	Context doğru taşıyor mu?

Her görev Python, TypeScript, Go, Rust, Zig ve Arelang taslak syntax'ı ile karşılaştırılmalıdır. Ölçümler: prompt tokeni, output tokeni, compile error sayısı, auto-fix tur sayısı, test pass oranı, benchmark latency, allocation count ve security lint violation sayısıdır.

Ek O - Compiler CLI ve geliştirici deneyimi

```
are new users-api --target server
are fmt
are check --json
are test
are run
are build --release
are bench
are audit --capabilities --unsafe --secrets
are openapi export --out openapi.json
are doctor
```

CLI komutları az ve tahmin edilebilir olmalıdır. check komutu hızlı çalışmalı ve codegen yapmadan type, region, capability ve diagnostics kontrollerini tamamlamalıdır. AI-agent loop için en kritik komut are check --json olacaktır. Bu komut stabil schema üretmelidir.

Komut	Görev
are check	Derlemeden tüm statik kontrolleri yapar.
are test	Unit/integration testleri çalıştırır.
are audit	Security, unsafe, capability raporu üretir.
are bench	Benchmark ve allocation raporu.
are doctor	Toolchain ve dependency sorunlarını açıklar.

Ek P - Örnek auto-fix döngüsü

AI-agent kullanımında ideal döngü şöyledir: model kod üretir, arefmt çalışır, are check --json çalışır, diagnostics modele geri verilir, model patch üretir, testler çalışır. Bu döngüde compiler'ın dili sade ve structured olması kritiktir.

```
User prompt
-> LLM generates src/users.are
-> are fmt
-> are check --json
-> diagnostics summarized to LLM
-> LLM emits patch
-> are test
-> are audit
-> are build --release
```

Bu döngüde modelden "neden hata oldu?" diye serbest açıklama istemek yerine diagnostic fix seçeneklerini kullanmak daha güvenilirdir. Compiler mümkünse machine-applicable edit önermelidir. Ancak güvenlik etkisi olan fixler otomatik uygulanmamalıdır; örneğin unsafe ekleme veya capability genişletme insan onayı gerektirir.

Ek Q - Anti-feature karar kayıtları

Her reddedilen özellik için kısa karar kaydı tutulmalıdır. Bu, ileride dil büyürken aynı tartışmaların tekrar edilmesini önler.

ID	Özellik	Karar	Gerekçe
AF-001	Exceptions	Rejected	Gizli control flow ve backend error mapping karmaşası.
AF-002	Null pointer	Rejected	Option ile daha güvenli model.
AF-003	Text macros	Rejected	Parser, diagnostics ve security belirsizliği.
AF-004	Unrestricted templates	Rejected for v0/v1	Error mesajları ve compile time riski.
AF-005	Operator overloading	Deferred	Sadece numeric/vector için ileride sınırlı olabilir.
AF-006	GC as default	Rejected	Latency ve runtime hedeflerine aykırı.
AF-007	Header files	Rejected	Build karmaşası ve AI hata riski.
AF-008	Reflection framework	Rejected	Runtime magic ve hidden behavior.

Ek R - İlk ürünleşme wedge'i

Dil tek başına pazara çıkmaz. İlk wedge, AI tarafından backend üreten bir platformun hedef dili olabilir. Kullanıcı doğal dille backend ister; sistem Arelang kodu üretir; compiler ve audit katmanı güvenliği sağlar; binary deploy edilir. Böylece dilin adoption problemi, ürünün sağladığı değerle aşılar.

Wedge	Neden uygun
Generated backend endpoint	AI kod üretimi için ana kullanım.
Sandboxed workflow plugin	Kullanıcı scriptleri güvenli çalışır.
Deployment agent	Küçük native binary avantajı.
High-performance worker	Queue ve batch processing.
Policy/audit engine	Security posture raporu.

Bu wedge özellikle Staçe benzeri bir sistemle uyumludur: kullanıcı backend mimarisi ister, platform codegen yapar, compiler hataları düzeltme döngüsüne girer, standard library auth/billing/observability sağlar, deployment tek binary ile yapılır. Dilin amacı developer'ı zorlamak değil, generated backendin kalitesini yükseltmektir.

Ek S - Spesifikasyon yazım sırası

Dil geliştirmeye başlamadan önce tüm spec'i kusursuz yazmaya çalışmak yanlış olur. Fakat bazı belgeler olmadan compiler yazmaya başlamak da dağınıklık yaratır. Aşağıdaki sıra yeterli denge sağlar.

Belge	İçerik
spec/syntax.md	Token, keyword, expression precedence, block structure
spec/types.md	Primitive, struct, enum, Option, Result, opaque type
spec/memory.md	Region, arena, heap, escape rules
spec/errors.md	Result, diagnostic codes, panic policy
spec/capabilities.md	Sandbox izinleri ve manifest
spec/stdlib-http.md	Service, route, Request, Response, middleware
spec/compiler.md	Pass pipeline ve IR overview
spec/ai-loop.md	JSON diagnostics ve agent workflow

Ek T - Nihai tasarım özeti

Arelang'ın ideal formu şu cümleyle özetlenebilir: backend servisleri ve sistem utilityleri için küçük, statik, nullsız, exception'sız, arena-first, C ABI uyumlu, structured diagnostics üreten, AI-agent döngüsünün birinci sınıf kabul eden native bir dil.

Başarı, çok özellik eklemekle değil; küçük bir alanı olağanüstü iyi çözmekle gelecektir. Eğer dil ilk 6 ayda güvenilir bir HTTP API, worker, validation, DB, observability ve audit deneyimi sunarsa; bunun üzerine generics, async, optimizing backend ve sandbox runtime güvenle inşa edilebilir.